| PRODUCTION | SEMANTIC RULES |
|---|---|
| $S \rightarrow$ **if** $E$ **then** $S_1$ | $E.true := newlabel;$<br>$E.false := S.next;$<br>$S_1.next := S.next;$<br>$S.code := E.code \parallel gen(E.true ':') \parallel S_1.code$ |
| $S \rightarrow$ **if** $E$ **then** $S_1$ **else** $S_2$ | $E.true := newlabel;$<br>$E.false := newlabel;$<br>$S_1.next := S.next;$<br>$S_2.next := S.next;$<br>$S.code := E.code \parallel gen(E.true ':') \parallel S_1.code \parallel gen('\texttt{goto}'\ S.next) \parallel gen(E.false ':') \parallel S_2.code$ |
| $S \rightarrow$ **while** $E$ **do** $S_1$ | $S.begin := newlabel;$<br>$E.true := newlabel;$<br>$E.false := S.next;$<br>$S_1.next := S.begin;$<br>$S.code := gen(S.begin ':') \parallel E.code \parallel gen(E.true ':') \parallel S_1.code \parallel gen('\texttt{goto}'\ S.begin)$ |

**Fig. 8.23.** Syntax-directed definition for flow-of-control statements.

We discuss the translation of flow-of-control statements in more detail in Section 8.6 where an alternative method, called "backpatching," emits code for such statements in one pass.

### Control-Flow Translation of Boolean Expressions

We now discuss *E.code*, the code produced for the boolean expressions $E$ in Fig. 8.23. As we have indicated, $E$ is translated into a sequence of three-address statements that evaluates $E$ as a sequence of conditional and unconditional jumps to one of two locations: *E.true*, the place control is to reach if $E$ is true, and *E.false*, the place control is to reach if $E$ is false.

The basic idea behind the translation is the following. Suppose $E$ is of the form `a < b`. Then the generated code is of the form

```
if a < b goto E.true
goto E.false
```

Suppose $E$ is of the form $E_1$ **or** $E_2$. If $E_1$ is true, then we immediately know that $E$ itself is true, so $E_1.true$ is the same as *E.true*. If $E_1$ is false, then

$E_2$ must be evaluated, so we make $E_1.false$ be the label of the first statement in the code for $E_2$. The true and false exits of $E_2$ can be made the same as the true and false exits of $E$, respectively.

Analogous considerations apply to the translation of $E_1$ **and** $E_2$. No code is needed for an expression $E$ of the form **not** $E_1$: We just interchange the true and false exits of $E_1$ to get the true and false exits of $E$. A syntax-directed definition that generates three-address code for boolean expressions in this manner is shown in Fig. 8.24. Note that the *true* and *false* attributes are inherited.

| PRODUCTION | SEMANTIC RULES |
|---|---|
| $E \rightarrow E_1$ **or** $E_2$ | $E_1.true := E.true$;<br>$E_1.false := newlabel$;<br>$E_2.true := E.true$;<br>$E_2.false := E.false$;<br>$E.code := E_1.code \parallel gen(E_1.false\text{':'}) \parallel E_2.code$ |
| $E \rightarrow E_1$ **and** $E_2$ | $E_1.true := newlabel$;<br>$E_1.false := E.false$;<br>$E_2.true := E.true$;<br>$E_2.false := E.false$;<br>$E.code := E_1.code \parallel gen(E_1.true\text{':'}) \parallel E_2.code$ |
| $E \rightarrow$ **not** $E_1$ | $E_1.true := E.false$;<br>$E_1.false := E.true$;<br>$E.code := E_1.code$ |
| $E \rightarrow$ ( $E_1$ ) | $E_1.true := E.true$;<br>$E_1.false := E.false$;<br>$E.code := E_1.code$ |
| $E \rightarrow$ **id**$_1$ **relop** **id**$_2$ | $E.code := gen(\text{'if'}\ \mathbf{id}_1.place\ \mathbf{relop}.op\ \mathbf{id}_2.place\ \text{'goto'}\ E.true) \parallel$<br>$gen(\text{'goto'}\ E.false)$ |
| $E \rightarrow$ **true** | $E.code := gen(\text{'goto'}\ E.true)$ |
| $E \rightarrow$ **false** | $E.code := gen(\text{'goto'}\ E.false)$ |

**Fig. 8.24.** Syntax-directed definition to produce three-address code for booleans.

**Example 8.4.** Let us again consider the expression

```
a < b or c < d and e < f
```

Suppose the true and false exits for the entire expression have been set to `Ltrue` and `Lfalse`. Then using the definition in Fig. 8.24 we would obtain the following code:

```
    if a < b goto Ltrue
    goto L1
L1: if c < d goto L2
    goto Lfalse
L2: if e < f goto Ltrue
    goto Lfalse
```

Note that the code generated is not optimal, in that the second statement can be eliminated without changing the value of the code. Redundant instructions of this form can be subsequently removed by a simple peephole optimizer (see Chapter 9). Another approach that avoids generating these redundant jumps is to translate a relational expression of the form $\mathbf{id}_1 < \mathbf{id}_2$ into the statement `if` $\mathbf{id}_1 \geq \mathbf{id}_2$ `goto` *E.false* with the presumption that when the relation is true we fall through the code. □

**Example 8.5.** Consider the statement

```
while a < b do
    if c < d then
        x := y + z
    else
        x := y - z
```

The syntax-directed definition above, coupled with schemes for assignment statements and boolean expressions, would produce the following code:

```
L1: if a < b goto L2
    goto Lnext
L2: if c < d goto L3
    goto L4
L3: t1 := y + z
    x := t1
    goto L1
L4: t2 := y - z
    x := t2
    goto L1
Lnext:
```

We note that the first two gotos can be eliminated by changing the directions of the tests. This type of local transformation can be done by peephole optimization discussed in Chapter 9. □

### Mixed-Mode Boolean Expressions

It is important to realize that we have simplified the grammar for boolean expressions. In practice, boolean expressions often contain arithmetic subexpressions as in `(a+b) < c`. In languages where false has the numerical value 0 and true the value 1, `(a < b) + (b < a)` can even be considered an arithmetic expression, with value 0 if `a` and `b` have the same value, and 1 otherwise.

The method of representing boolean expressions by jumping code can still be used, even if arithmetic expressions are represented by code to compute their value. For example, consider the representative grammar

$$E \rightarrow E + E \mid E \textbf{ and } E \mid E \textbf{ relop } E \mid \textbf{id}$$

We may suppose that $E + E$ produces an integer arithmetic result (the inclusion of real or other arithmetic types makes matters more complicated but adds nothing to the instructive value of this example), while expressions $E$ **and** $E$ and $E$ **relop** $E$ produce boolean values represented by flow of control. Expression $E$ **and** $E$ requires both arguments to be boolean, but the operations + and **relop** take either type of argument, including mixed ones. $E \rightarrow$ **id** is also deemed arithmetic, although we could extend this example by allowing boolean identifiers.

To generate code in this situation, we can use a synthesized attribute *E.type*, which will be either *arith* or *bool* depending on the type of $E$. $E$ will have inherited attributes *E.true* and *E.false* for boolean expressions and synthesized attribute *E.place* for arithmetic expressions. Part of the semantic rule for $E \rightarrow E_1 + E_2$ is shown in Fig. 8.25.

```
E.type := arith;
if E1.type = arith and E2.type = arith then begin
      /* normal arithmetic add */
      E.place := newtemp;
      E.code := E1.code || E2.code ||
          gen(E.place ':=' E1.place '+' E2.place)
end
else if E1.type = arith and E2.type = bool then begin
      E.place := newtemp;
      E2.true := newlabel;
      E2.false := newlabel;
      E.code := E1.code || E2.code ||
          gen(E2.true ':' E.place ':=' E1.place + 1) ||
          gen('goto' nextstat + 1) ||
          gen(E2.false ':' E.place ':=' E1.place)
else if ···
```

**Fig. 8.25.** Semantic rule for production $E \rightarrow E_1 + E_2$.

In the mixed-mode case, we generate the code for $E_1$, then $E_2$, followed by the three statements:

```
E2.true:  E.place := E1.place + 1
          goto nextstat + 1
E2.false: E.place := E1.place
```

The first statement computes the value $E_1 + 1$ for $E$ when $E_2$ is true, the third the value $E_1$ for $E$ when $E_2$ is false. The second statement is a jump over the third. The semantic rules for the remaining cases and the other productions are quite similar, and we leave them as exercises.

## 8.5 CASE STATEMENTS

The "switch" or "case" statement is available in a variety of languages; even the Fortran computed and assigned `goto`'s can be regarded as varieties of the switch statement. Our switch-statement syntax is shown in Fig. 8.26.

```
switch expression
    begin
        case value:    statement
        case value:    statement
                . . .
        case value:    statement
        default:       statement
    end
```

**Fig. 8.26.** Switch-statement syntax.

There is a selector expression, which is to be evaluated, followed by $n$ constant values that the expression might take, perhaps including a *default* "value," which always matches the expression if no other value does. The intended translation of a switch is code to:

1. Evaluate the expression.
2. Find which value in the list of cases is the same as the value of the expression. Recall that the default value matches the expression if none of the values explicitly mentioned in cases does.
3. Execute the statement associated with the value found.

Step (2) is an $n$-way branch, which can be implemented in one of several ways. If the number of cases is not too great, say 10 at most, then it is reasonable to use a sequence of conditional `goto`'s, each of which tests for an individual value and transfers to the code for the corresponding statement.

A more compact way to implement this sequence of conditional `goto`'s is to create a table of pairs, each pair consisting of a value and a label for the code of the corresponding statement. Code is generated to place at the end of this table the value of the expression itself, paired with the label for the default statement. A simple loop can be generated by the compiler to compare the value of the expression with each value in the table, being assured that if no other match is found, the last (default) entry is sure to match.

If the number of values exceeds 10 or so, it is more efficient to construct a

hash table (see Section 7.6) for the values, with the labels of the various statements as entries. If no entry for the value possessed by the switch expression is found, a jump to the default statement can be generated.

There is a common special case in which an even more efficient implementation of the $n$-way branch exists. If the values all lie in some small range, say $i_{min}$ to $i_{max}$, and the number of different values is a reasonable fraction of $i_{max}-i_{min}$, then we can construct an array of labels, with the label of the statement for value $j$ in the entry of the table with offset $j-i_{min}$ and the label for the default in entries not filled otherwise. To perform the switch, evaluate the expression to obtain the value $j$, check that it is in the range $i_{min}$ to $i_{max}$ and transfer indirectly to the table entry at offset $j-i_{min}$. For example, if the expression is of type character, a table of, say, 128 entries (depending on the character set) may be created and transferred through with no range testing.

### Syntax-Directed Translation of Case Statements

Consider the following switch statement.

```
switch E
    begin
        case V1:    S1
        case V2:    S2
                . . .
        case Vn-1:  Sn-1
        default:    Sn
    end
```

With a syntax-directed translation scheme, it is convenient to translate this case statement into intermediate code that has the form of Fig. 8.27.

The tests all appear at the end so that a simple code generator can recognize the multiway branch and generate efficient code for it, using the most appropriate implementation suggested at the beginning of this section. If we generate the more straightforward sequence shown in Fig 8.28, the compiler would have to do extensive analysis to find the most efficient implementation. Note that it is inconvenient to place the branching statements at the beginning, because the compiler could not then emit code for each of the $S_i$'s as it saw them.

To translate into the form of Fig. 8.27, when we see the keyword **switch**, we generate two new labels `test` and `next`, and a new temporary `t`. Then as we parse the expression $E$, we generate code to evaluate $E$ into `t`. After processing $E$, we generate the jump `goto test`.

Then as we see each **case** keyword, we create a new label $L_i$ and enter it into the symbol table. We place on a stack, used only to store cases, a pointer to this symbol-table entry and the value $V_i$ of the case constant. (If this switch is embedded in one of the statements internal to another switch, we place a marker on the stack to separate cases for the interior switch from those for the outer switch.)

```
        code to evaluate E into t
        goto test
L1:     code for S1
        goto next
L2:     code for S2
        goto next
          . . .
Ln-1:   code for Sn-1
        goto next
Ln:     code for Sn
        goto next
test:   if t = V1 goto L1
        if t = V2 goto L2
          . . .
        if t = Vn-1 goto Ln-1
        goto Ln
next:
```

**Fig. 8.27.** Translation of a case statement.

```
        code to evaluate E into t
        if t ≠ V1 goto L1
        code for S1
        goto next
L1:     if t ≠ V2 goto L2
        code for S2
        goto next
L2:
          . . .
Ln-2:   if t ≠ Vn-1 goto Ln-1
        code for Sn-1
        goto next
Ln-1:   code for Sn
next:
```

**Fig. 8.28.** Another translation of a case statement.

We process each statement **case** $V_i$: $S_i$ by emitting the newly created label $L_i$, followed by the code for $S_i$, followed by the jump `goto next`. Then when the keyword **end** terminating the body of the switch is found, we are ready to generate the code for the $n$-way branch. Reading the pointer-value pairs on the case stack from the bottom to the top, we can generate a sequence of three-address statements of the form

```
case V1 L1
case V2 L2

    . . .

case Vn-1 Ln-1
case t Ln
label next
```

where t is the name holding the value of the selector expression $E$, and $L_n$ is the label for the default statement. The `case` $V_i$ $L_i$ three-address statement is a synonym for `if` t = $V_i$ `goto` $L_i$ in Fig. 8.27, but the `case` is easier for the final code generator to detect as a candidate for special treatment. At the code-generation phase, these sequences of `case` statements can be translated into an $n$-way branch of the most efficient type, depending on how many there are and whether the values fall into a small range.

## 8.6 BACKPATCHING

The easiest way to implement the syntax-directed definitions in Section 8.4 is to use two passes. First, construct a syntax tree for the input, and then walk the tree in depth-first order, computing the translations given in the definition. The main problem with generating code for boolean expressions and flow-of-control statements in a single pass is that during one single pass we may not know the labels that control must go to at the time the jump statements are generated. We can get around this problem by generating a series of branching statements with the targets of the jumps temporarily left unspecified. Each such statement will be put on a list of goto statements whose labels will be filled in when the proper label can be determined. We call this subsequent filling in of labels *backpatching*.

In this section, we show how backpatching can be used to generate code for boolean expressions and flow-of-control statements in one pass. The translations we generate will be of the same form as those in Section 8.4, except for the manner in which we generate labels. For specificity, we generate quadruples into a quadruple array. Labels will be indices into this array. To manipulate lists of labels, we use three functions:

1. *makelist*($i$) creates a new list containing only $i$, an index into the array of quadruples; *makelist* returns a pointer to the list it has made.
2. *merge*($p_1$, $p_2$) concatenates the lists pointed to by $p_1$ and $p_2$, and returns a pointer to the concatenated list.
3. *backpatch*($p$, $i$) inserts $i$ as the target label for each of the statements on the list pointed to by $p$.

### Boolean Expressions

We now construct a translation scheme suitable for producing quadruples for boolean expressions during bottom-up parsing. We insert a marker nonterminal $M$ into the grammar to cause a semantic action to pick up, at appropriate times, the index of the next quadruple to be generated. The grammar we use is the following:

(1) $E \rightarrow E_1$ **or** $M\ E_2$
(2) $\quad | \ E_1$ **and** $M\ E_2$
(3) $\quad | \ $ **not** $E_1$
(4) $\quad | \ (\ E_1\ )$
(5) $\quad | \ \mathbf{id}_1$ **relop** $\mathbf{id}_2$
(6) $\quad | \ $ **true**
(7) $\quad | \ $ **false**
(8) $M \rightarrow \epsilon$

Synthesized attributes *truelist* and *falselist* of nonterminal $E$ are used to generate jumping code for boolean expressions. As code is generated for $E$, jumps to the true and false exits are left incomplete, with the label field unfilled. These incomplete jumps are placed on lists pointed to by $E.truelist$ and $E.falselist$, as appropriate.

The semantic actions reflect the considerations mentioned above. Consider the production $E \rightarrow E_1$ **and** $M\ E_2$. If $E_1$ is false, then $E$ is also false, so the statements on $E_1.falselist$ become part of $E.falselist$. If $E_1$ is true, however, we must next test $E_2$, so the target for the statements $E_1.truelist$ must be the beginning of the code generated for $E_2$. This target is obtained using the marker nonterminal $M$. Attribute $M.quad$ records the number of the first statement of $E_2.code$. With the production $M \rightarrow \epsilon$ we associate the semantic action

$$\{\ M.quad := nextquad\ \}$$

The variable *nextquad* holds the index of the next quadruple to follow. This value will be backpatched onto the $E_1.truelist$ when we have seen the remainder of the production $E \rightarrow E_1$ **and** $M\ E_2$. The translation scheme is as follows.

(1) $E \rightarrow E_1$ **or** $M\ E_2$ { *backpatch* $(E_1.falselist, M.quad)$;
$E.truelist := merge(E_1.truelist, E_2.truelist)$;
$E.falselist := E_2.falselist$ }

(2) $E \rightarrow E_1$ **and** $M\ E_2$ { *backpatch* $(E_1.truelist, M.quad)$;
$E.truelist := E_2.truelist$;
$E.falselist := merge(E_1.falselist, E_2.falselist)$ }

(3) $E \rightarrow$ **not** $E_1$ { *E.truelist* := $E_1$.*falselist*;
*E.falselist* := $E_1$.*truelist* }

(4) $E \rightarrow$ ( $E_1$ ) { *E.truelist* := $E_1$.*truelist*;
*E.falselist* := $E_1$.*falselist* }

(5) $E \rightarrow$ **id**$_1$ **relop** **id**$_2$ { *E.truelist* := *makelist*(*nextquad*);
*E.falselist* := *makelist*(*nextquad* + 1);
*emit*('`if`' **id**$_1$.*place* **relop**.*op* **id**$_2$.*place* '`goto _`')
*emit*('`goto _`') }

(6) $E \rightarrow$ **true** { *E.truelist* := *makelist*(*nextquad*);
*emit*('`goto _`') }

(7) $E \rightarrow$ **false** { *E.falselist* := *makelist*(*nextquad*);
*emit*('`goto _`') }

(8) $M \rightarrow \epsilon$ { *M.quad* := *nextquad* }

For simplicity, semantic action (5) generates two statements, a conditional goto and an unconditional one. Neither has its target filled in. The index of the first generated statement is made into a list, and *E.truelist* is given a pointer to that list. The second generated statement `goto _` is also made into a list and given to *E.falselist*.

**Example 8.6.** Consider again the expression `a < b or c < d and e < f`. An annotated parse tree is shown in Fig. 8.29. The actions are performed during a depth-first traversal of the tree. Since all actions appear at the ends of right sides, they can be performed in conjunction with reductions during a bottom-up parse. In response to the reduction of `a < b` to *E* by production (5), the two quadruples

```
100: if a < b goto _
101: goto _
```

are generated. (We again arbitrarily start statement numbers at 100.) The marker nonterminal *M* in the production $E \rightarrow E_1$ **or** $M$ $E_2$ records the value of *nextquad*, which at this time is 102. The reduction of `c < d` to *E* by production (5) generates the quadruples

```
102: if c < d goto _
103: goto _
```

We have now seen $E_1$ in the production $E \rightarrow E_1$ **and** $M$ $E_2$. The marker nonterminal in this production records the current value of *nextquad*, which is now 104. Reducing `e < f` into *E* by production (5) generates

```
104: if e < f goto _
105: goto _
```

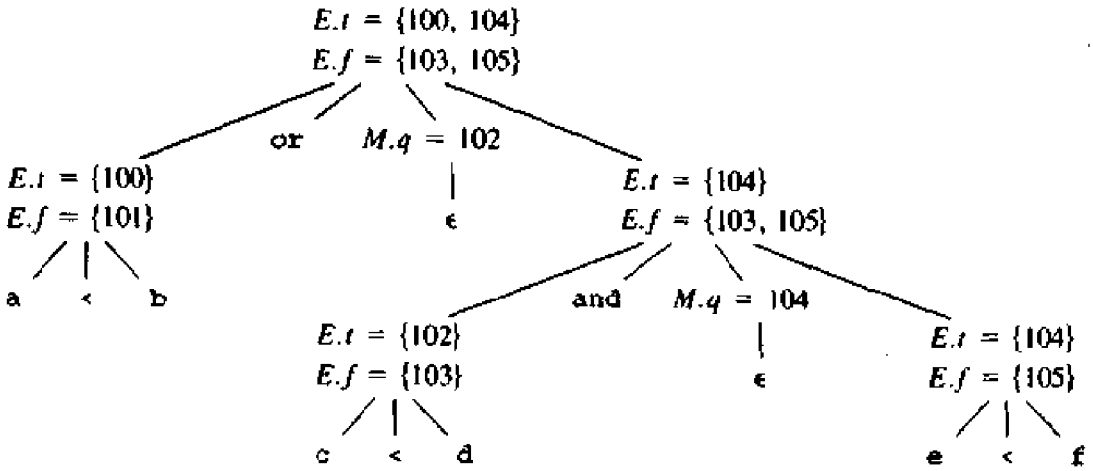


**Fig. 8.29.** Annotated parse tree for `a < b or c < d and e < f`.

We now reduce by $E \rightarrow E_1$ **and** $M\ E_2$. The corresponding semantic action calls *backpatch*({102},104) where {102} as argument denotes a pointer to the list containing only 102, that list being the one pointed to by $E_1$.*truelist*. This call to *backpatch* fills in 104 in statement 102. The six statements generated so far are thus:

```
100:  if a < b goto _
101:  goto _
102:  if c < d goto 104
103:  goto _
104:  if e < f goto _
105:  goto _
```

The semantic action associated with the final reduction by $E \rightarrow E_1$ **or** $M\ E_2$ calls *backpatch*({101},102) which leaves the statements looking like:

```
100:  if a < b goto _
101:  goto 102
102:  if c < d goto 104
103:  goto _
104:  if e < f goto _
105:  goto _
```

The entire expression is true if and only if the goto's of statements 100 or 104 are reached, and is false if and only if the goto's of statements 103 or 105 are reached. These instructions will have their targets filled in later in the compilation, when it is seen what must be done depending on the truth or falsehood of the expression. □

### Flow-of-Control Statements

We now show how backpatching can be used to translate flow-of-control statements in one pass. As above, we fix our attention on the generation of quadruples, and the notation regarding translation field names and list-handling procedures from that section carries over to this section as well. As a larger example, we develop a translation scheme for statements generated by the following grammar:

(1) $S \rightarrow$ **if** $E$ **then** $S$
(2) | **if** $E$ **then** $S$ **else** $S$
(3) | **while** $E$ **do** $S$
(4) | **begin** $L$ **end**
(5) | $A$
(6) $L \rightarrow L$ ; $S$
(7) | $S$

Here $S$ denotes a statement, $L$ a statement list, $A$ an assignment statement, and $E$ a boolean expression. Note that there must be other productions, such as those for assignment statements. The productions given, however, will be sufficient to illustrate the techniques used to translate flow-of-control statements.

We use the same structure of code for if-then, if-then-else, and while-do statements as in Section 8.4. We make the tacit assumption that the code that follows a given statement in execution also follows it physically in the quadruple array. If that is not true, an explicit jump must be provided.

Our general approach will be to fill in the jumps out of statements when their targets are found. Not only do boolean expressions need two lists of jumps that occur when the expression is true and when it is false, but statements also need lists of jumps (given by attribute *nextlist*) to the code that follows them in the execution sequence.

### Scheme to Implement the Translation

We now describe a syntax-directed translation scheme to generate translations for the flow-of-control constructs given above. The nonterminal $E$ has two attributes $E.truelist$ and $E.falselist$, as above. $L$ and $S$ each also need a list of unfilled quadruples that must eventually be completed by backpatching. These lists are pointed to by the attributes $L.nextlist$ and $S.nextlist$. $S.nextlist$ is a pointer to a list of all conditional and unconditional jumps to the quadruple following the statement $S$ in execution order, and $L.nextlist$ is defined similarly.

In the code layout for $S \rightarrow$ **while** $E$ **do** $S_1$ in Fig. 8.22(c), there are labels $S.begin$ and $E.true$ marking the beginning of the code for the complete statement $S$ and the body $S_1$. The two occurrences of the marker nonterminal $M$ in the following production record the quadruple numbers of these positions:

$$S \rightarrow \textbf{while}\ M_1\ E\ \textbf{do}\ M_2\ S_1$$

Again, the only production for $M$ is $M \rightarrow \epsilon$ with an action setting attribute *M.quad* to the number of the next quadruple. After the body $S_1$ of the while statement is executed, control flows to the beginning. Therefore, when we reduce **while** $M_1$ $E$ **do** $M_2$ $S_1$ to $S$, we backpatch $S_1$.*nextlist* to make all targets on that list be $M_1$.*quad*. An explicit jump to the beginning of the code for $E$ is appended after the code for $S_1$ because control may also "fall out the bottom." *E.truelist* is backpatched to go to the beginning of $S_1$ by making jumps on *E.truelist* go to $M_2$.*quad*.

A more compelling argument for using *S.nextlist* and *L.nextlist* comes when code is generated for the conditional statement **if** $E$ **then** $S_1$ **else** $S_2$. If control "falls out the bottom" of $S_1$, as when $S_1$ is an assignment, we must include at the end of the code for $S_1$ a jump over the code for $S_2$. We use another marker nonterminal to introduce this jump after $S_1$. Let nonterminal $N$ be this marker with production $N \rightarrow \epsilon$. $N$ has attribute *N.nextlist*, which will be a list consisting of the quadruple number of the statement `goto _` that is generated by the semantic rule for $N$. We now give the semantic rules for the revised grammar.

(1) $S \rightarrow$ **if** $E$ **then** $M_1$ $S_1$ $N$ **else** $M_2$ $S_2$

{ *backpatch*(*E.truelist*, $M_1$.*quad*);
*backpatch*(*E.falselist*, $M_2$.*quad*);
*S.nextlist* := *merge*($S_1$.*nextlist*, *merge*(*N.nextlist*, $S_2$.*nextlist*)) }

We backpatch the jumps when $E$ is true to the quadruple $M_1$.*quad*, which is the beginning of the code for $S_1$. Similarly, we backpatch jumps when $E$ is false to go to the beginning of the code for $S_2$. The list *S.nextlist* includes all jumps out of $S_1$ and $S_2$, as well as the jump generated by $N$.

(2) $N \rightarrow \epsilon$ { *N.nextlist* := *makelist*(*nextquad*);
*emit*('`goto _`') }

(3) $M \rightarrow \epsilon$ { *M.quad* := *nextquad* }

(4) $S \rightarrow$ **if** $E$ **then** $M$ $S_1$ { *backpatch*(*E.truelist*, *M.quad*);
*S.nextlist* := *merge*(*E.falselist*, $S_1$.*nextlist*) }

(5) $S \rightarrow$ **while** $M_1$ $E$ **do** $M_2$ $S_1$ { *backpatch*($S_1$.*nextlist*, $M_1$.*quad*);
*backpatch*(*E.truelist*, $M_2$.*quad*);
*S.nextlist* := *E.falselist*
*emit*('`goto`' $M_1$.*quad*) }

(6) $S \rightarrow$ **begin** $L$ **end** { *S.nextlist* := *L.nextlist* }

(7) $S \rightarrow A$ { *S.nextlist* := **nil** }

The assignment *S.nextlist* := **nil** initializes *S.nextlist* to an empty list.

(8) $L \rightarrow L_1$ ; $M$ $S$ { *backpatch*($L_1$.*nextlist*, *M.quad*);
*L.nextlist* := *S.nextlist* }

The statement following $L_1$ in order of execution is the beginning of $S$. Thus the $L_1.nextlist$ list is backpatched to the beginning of the code for $S$, which is given by $M.quad$.

(9) $L \rightarrow S$ { $L.nextlist := S.nextlist$ }

Note that no new quadruples are generated anywhere in these semantic rules except for rules (2) and (5). All other code is generated by the semantic actions associated with assignment statements and expressions. What the flow of control does is cause the proper backpatching so that the assignments and boolean expression evaluations will connect properly.

### Labels and Gotos

The most elementary programming language construct for changing the flow of control in a program is the label and goto. When a compiler encounters a statement like `goto L`, it must check that there is exactly one statement with label `L` in the scope of this `goto` statement. If the label has already appeared, either in a label-declaration statement or as the label of some source statement, then the symbol table will have an entry giving the compiler-generated label for the first three-address instruction associated with the source statement labeled `L`. For the translation we generate a `goto` three-address statement with that compiler-generated label as target.

When a label `L` is encountered for the first time in the source program, either in a declaration or as the target of a forward goto, we enter `L` into the symbol table and generate a symbolic label for `L`.

## 8.7 PROCEDURE CALLS

The procedure[6] is such an important and frequently used programming construct that it is imperative for a compiler to generate good code for procedure calls and returns. The run-time routines that handle procedure argument passing, calls, and returns are part of the run-time support package. We discussed the different kinds of mechanisms needed to implement the run-time support package in Chapter 7. In this section, we discuss the code that is typically generated for procedure calls and returns.

Let us consider a grammar for a simple procedure call statement.

(1) $S \rightarrow$ **call id** ( *Elist* )
(2) *Elist* $\rightarrow$ *Elist* , $E$
(3) *Elist* $\rightarrow E$

[6] We use the term procedure to include function. A function is a procedure that returns a value.

### Calling Sequences

As discussed in Chapter 7, the translation for a call includes a calling sequence, a sequence of actions taken on entry to and exit from each procedure. While calling sequences differ, even for implementations of the same language, the following actions typically take place:

When a procedure call occurs, space must be allocated for the activation record of the called procedure. The arguments of the called procedure must be evaluated and made available to the called procedure in a known place. Environment pointers must be established to enable the called procedure to access data in enclosing blocks. The state of the calling procedure must be saved so it can resume execution after the call. Also saved in a known place is the return address, the location to which the called routine must transfer after it is finished. The return address is usually the location of the instruction that follows the call in the calling procedure. Finally, a jump to the beginning of the code for the called procedure must be generated.

When a procedure returns, several actions also must take place. If the called procedure is a function, the result must be stored in a known place. The activation record of the calling procedure must be restored. A jump to the calling procedure's return address must be generated.

There is no exact division of the run-time tasks between the calling and called procedure. Often the source language, the target machine, and the operating system impose requirements that favor one solution over another.

### A Simple Example

Let us consider a simple example in which parameters are passed by reference and storage is statically allocated. In this situation, we can use the `param` statements themselves as placeholders for the arguments. The called procedure is passed a pointer in a register to the first of the `param` statements, and can obtain pointers to any of its arguments by using the proper offset from this base pointer. When generating three-address code for this type of call, it is sufficient to generate the three-address statements needed to evaluate those arguments that are expressions other than simple names, then follow them by a list of `param` three-address statements, one for each argument. If we do not want to mix the argument-evaluating statements with the `param` statements, we shall have to save the value of *E.place*, for each expression $E$ in **id**$(E, E, \ldots, E)$.[7]

A convenient data structure in which to save these values is a queue, a first-in first-out list. Our semantic routine for *Elist* → *Elist* , $E$ will include a step to store *E.place* on a queue *queue*. Then, the semantic routine for

[7] If parameters are passed to the called procedure by putting them on a stack, as would normally be the case for dynamically allocated data, there is no reason not to mix evaluation and `param` statements. The `param` statement is replaced at code-generation time by code to push a parameter on the stack.

$S \rightarrow$ **call id** ( *Elist* ) will generate a `param` statement for each item on *queue*, causing these statements to follow the statements evaluating the argument expressions. Those statements were generated when the arguments themselves were reduced to $E$. The following syntax-directed translation incorporates these ideas.

(1) $S \rightarrow$ **call id** ( *Elist* )
{ **for** each item $p$ on *queue* **do**
*emit*('`param`' $p$);
*emit*('`call`' **id**.*place*) }

The code for $S$ is the code for *Elist*, which evaluates the arguments, followed by a `param` $p$ statement for each argument, followed by a **call** statement. A count of the number of parameters is not generated with the `call` statement but could be calculated in the same way we computed *Elist.ndim* in the previous section.

(2) *Elist* $\rightarrow$ *Elist* , $E$
{ append $E$.*place* to the end of *queue* }

(3) *Elist* $\rightarrow E$
{ initialize *queue* to contain only $E$.*place* }

Here *queue* is emptied and then gets a single pointer to the symbol table location for the name that denotes the value of $E$.

## EXERCISES

**8.1** Translate the arithmetic expression `a * - (b + c)` into
a) a syntax tree
b) postfix notation
c) three-address code.

**8.2** Translate the expression `-(a + b) * (c + d) + (a + b + c)` into
a) quadruples
b) triples
c) indirect triples.

**8.3** Translate the executable statements of the following C program

```
main()
{
    int i;
    int a[10];
    i = 1;
    while (i <= 10) {
        a[i] = 0; i = i + 1;
    }
}
```

into

a) a syntax tree
b) postfix notation
c) three-address code.

*8.4 Prove that if all operators are binary, then a string of operators and operands is a postfix expression if and only if (1) there is exactly one fewer operator than operands, and (2) every nonempty prefix of the expression has fewer operators than operands.

8.5 Modify the translation scheme in Fig. 8.11 for computing the types and relative addresses of declared names to allow lists of names instead of single names in declarations of the form $D \rightarrow \mathbf{id}\ ;\ T$.

8.6 The *prefix* form of an expression in which operator $\theta$ is applied to expressions $e_1, e_2, \ldots, e_k$ is $\theta p_1 p_2 \cdots p_k$, where $p_i$ is the prefix form of $e_i$.

a) Generate the prefix form of `a * - (b+c)`.

**b) Show that infix expressions cannot be translated into prefix form by translation schemes in which all actions are printing actions and all actions appear at the ends of right sides of productions.

c) Give a syntax-directed definition to translate infix expressions into prefix form. Which of the methods of Chapter 5 can you use?

8.7 Write a program to implement the syntax-directed definition for translating booleans to three-address code given in Fig. 8.24.

8.8 Modify the syntax-directed definition in Fig. 8.24 to generate code for the stack machine of Section 2.8.

8.9 The syntax-directed definition in Fig. 8.24 translates $E \rightarrow \mathbf{id}_1 < \mathbf{id}_2$ into the pair of statements

**if** $\mathbf{id}_1 < \mathbf{id}_2$ **goto** $\cdots$
**goto** $\cdots$

We could translate instead into the single statement

**if** $\mathbf{id}_1 \geq \mathbf{id}_2$ **goto** _

and fall through the code when $E$ is true. Modify the definition in Fig. 8.24 to generate code of this nature.

8.10 Write a program to implement the syntax-directed definition for flow-of-control statements given in Fig. 8.23.

8.11 Write a program to implement the backpatching algorithm given in Section 8.6.

8.12 Translate the following assignment statement into three-address code using the translation scheme in Section 8.3.

```
A[i,j] := B[i,j] + C[A[k,l]] + D[i+j]
```

***8.13** Some languages, such as PL/I, permit a list of names to be given a list of attributes and also permit declarations to be nested within one another. The following grammar abstracts the problem:

$$
\begin{aligned}
D &\rightarrow \textit{namelist attrlist} \\
&\mid \text{( } D \text{ ) } \textit{attrlist} \\
\textit{namelist} &\rightarrow \textbf{id} \text{ , } \textit{namelist} \\
&\mid \textbf{id} \\
\textit{attrlist} &\rightarrow A \textit{ attrlist} \\
&\mid A \\
A &\rightarrow \textbf{decimal} \mid \textbf{fixed} \mid \textbf{float} \mid \textbf{real}
\end{aligned}
$$

The meaning of $D \rightarrow ( D )$ *attrlist* is that all names mentioned in the declaration inside parentheses are given the attributes on *attrlist*, no matter how many levels of nesting there are. Note that a declaration of $n$ names and $m$ attributes may cause $nm$ pieces of information to be entered into the symbol table. Give a syntax-directed definition for declarations defined by this grammar.

**8.14** In C, the for statement has the following form:

```
for ( e1 ; e2 ; e3 ) stmt
```

Taking its meaning to be

```
e1;
while ( e2 ) {
    stmt;
    e3;
}
```

construct a syntax-directed definition to translate C-style for statements into three-address code.

**8.15** The Pascal standard defines the statement

**for** *v* := *initial* **to** *final* **do** *stmt*

to have the same meaning as the following code sequence:

```
begin
    t1 := initial; t2 := final;
    if t1 ≤ t2 then begin
        v := t1;
        stmt
        while v ≠ t2 do begin
            v := succ(v);
            stmt
        end
    end
end
```

a) Consider the following Pascal program:

```
program forloop(input, output);
    var i, initial, final: integer;
    begin
        read(initial, final);
        for i:= initial to final do
            writeln(i)
    end.
```

What behavior does this program have for *initial* = MAXINT − 5 and *final* = MAXINT, where MAXINT is the largest integer on the target machine.

*b) Construct a syntax-directed definition that generates correct three-address code for Pascal for-statements.

## BIBLIOGRAPHIC NOTES

UNCOL (for Universal Compiler Oriented Language) is a mythical universal intermediate language, sought since the mid 1950's. Given an UNCOL, the committee report by Strong et al. [1958] showed how compilers could be constructed by hooking a front end for a given source language with a back end for a given target language. The bootstrapping techniques given in the report are routinely used to retarget compilers (see Section 11.2). Steel [1961] contains an original proposal for UNCOL.

A retargetable compiler consists of one front end that can be put together with several back ends to implement a given language on several machines. Neliac is an early example of a language with a retargetable compiler (Huskey, Halstead, and McArthur [1960]) written in its own language. See also Richards [1971] for a description of a retargetable compiler for BCPL, Nori et al. [1981] for Pascal, and Johnson [1979] for C. Newey, Poole, and Waite [1972] apply the idea of changing the back end to a macro processor, a text editor, and a Basic compiler.

The UNCOL ideal of implementing $n$ languages on $m$ machines by writing $n$ front ends and $m$ back ends, as opposed to $n \times m$ distinct compilers, has been approached in a number of ways. One approach is to retro-fit a front end for a new language onto an existing compiler. Feldman [1979b] describes the addition of a Fortran 77 front end to the C compilers by Johnson [1979] and Ritchie [1979]. Compiler organizations designed to accomodate multiple front ends and back ends are described by Davidson and Fraser [1984b], Leverett et al. [1980], and Tanenbaum et al. [1983].

The terms "union" and "intersection" abstract machines used by Davidson and Fraser [1984b] highlight the role of the set of allowable operators in an intermediate representation. The instruction set and addressing modes of an intersection machine are limited, so the front end does not have to make many choices when it generates intermediate code. Union machines provide alternative ways of implementing source-level constructs. Since all of the alternatives

may not be implemented directly by all target machines, the richer instruction set of the union machine may allow dependence on the target machine to creep in. Similar remarks apply to other kinds of intermediate code, such as syntax trees and three-address code. Fraser and Hanson [1982] consider ways of expressing access to the run-time stack using machine-independent operations.

The implementation of Algol 60 is discussed in detail by Randell and Russell [1964] and Grau, Hill, and Langmaack [1967]. Freiburghouse [1969] discusses PL/I, Wirth [1971] Pascal, and Branquart et al. [1976] Algol 68.

Minker and Minker [1980] and Giegerich and Wilhelm [1978] discuss the generation of optimal code for boolean expressions. Exercise 8.15 is from Newey and Waite [1985].

CHAPTER 9

# Code Generation

The final phase in our compiler model is the code generator. It takes as input an intermediate representation of the source program and produces as output an equivalent target program, as indicated in Fig. 9.1. The code generation techniques presented in this chapter can be used whether or not an optimization phase occurs before code generation, as in some so-called "optimizing" compilers. Such a phase tries to transform the intermediate code into a form from which more efficient target code can be produced. We shall talk about code optimization in detail in the next chapter.

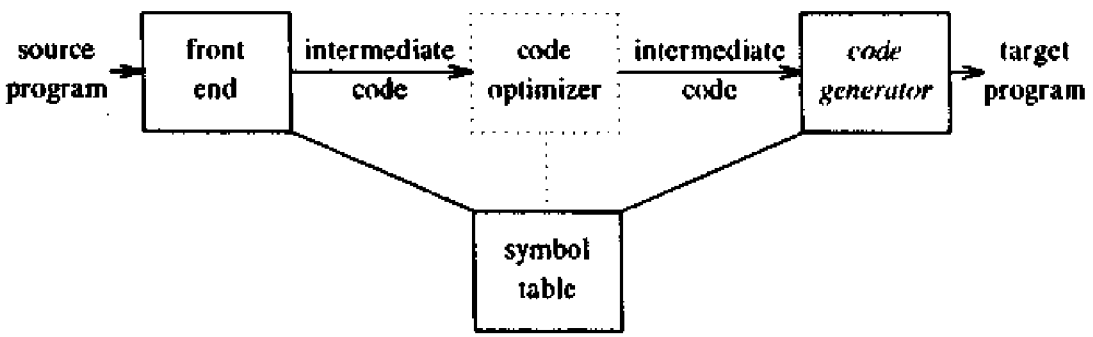


**Fig. 9.1.** Position of code generator.

The requirements traditionally imposed on a code generator are severe. The output code must be correct and of high quality, meaning that it should make effective use of the resources of the target machine. Moreover, the code generator itself should run efficiently.

Mathematically, the problem of generating optimal code is undecidable. In practice, we must be content with heuristic techniques that generate good, but not necessarily optimal, code. The choice of heuristics is important, in that a carefully designed code generation algorithm can easily produce code that is several times faster than that produced by a hastily conceived algorithm.

## 9.1 ISSUES IN THE DESIGN OF A CODE GENERATOR

While the details are dependent on the target language and the operating system, issues such as memory management, instruction selection, register allocation, and evaluation order are inherent in almost all code-generation problems. In this section, we shall examine the generic issues in the design of code generators.

### Input to the Code Generator

The input to the code generator consists of the intermediate representation of the source program produced by the front end, together with information in the symbol table that is used to determine the run-time addresses of the data objects denoted by the names in the intermediate representation.

As we noted in the previous chapter, there are several choices for the intermediate language, including: linear representations such as postfix notation, three-address representations such as quadruples, virtual machine representations such as stack machine code, and graphical representations such as syntax trees and dags. Although the algorithms in this chapter are couched in terms of three-address code, trees, and dags, many of the techniques also apply to the other intermediate representations.

We assume that prior to code generation the front end has scanned, parsed, and translated the source program into a reasonably detailed intermediate representation, so the values of names appearing in the intermediate language can be represented by quantities that the target machine can directly manipulate (bits, integers, reals, pointers, etc.). We also assume that the necessary type checking has taken place, so type-conversion operators have been inserted wherever necessary and obvious semantic errors (e.g., attempting to index an array by a floating-point number) have already been detected. The code generation phase can therefore proceed on the assumption that its input is free of errors. In some compilers, this kind of semantic checking is done together with code generation.

### Target Programs

The output of the code generator is the target program. Like the intermediate code, this output may take on a variety of forms: absolute machine language, relocatable machine language, or assembly language.

Producing an absolute machine-language program as output has the advantage that it can be placed in a fixed location in memory and immediately executed. A small program can be compiled and executed quickly. A number of "student-job" compilers, such as WATFIV and PL/C, produce absolute code.

Producing a relocatable machine-language program (object module) as output allows subprograms to be compiled separately. A set of relocatable object modules can be linked together and loaded for execution by a linking loader. Although we must pay the added expense of linking and loading if we produce

relocatable object modules, we gain a great deal of flexibility in being able to compile subroutines separately and to call other previously compiled programs from an object module. If the target machine does not handle relocation automatically, the compiler must provide explicit relocation information to the loader to link the separately compiled program segments.

Producing an assembly-language program as output makes the process of code generation somewhat easier. We can generate symbolic instructions and use the macro facilities of the assembler to help generate code. The price paid is the assembly step after code generation. Because producing assembly code does not duplicate the entire task of the assembler, this choice is another reasonable alternative, especially for a machine with a small memory, where a compiler must use several passes. In this chapter, we use assembly code as the target language for readability. However, we should emphasize that as long as addresses can be calculated from the offsets and other information stored in the symbol table, the code generator can produce relocatable or absolute addresses for names just as easily as symbolic addresses.

### Memory Management

Mapping names in the source program to addresses of data objects in run-time memory is done cooperatively by the front end and the code generator. In the last chapter, we assumed that a name in a three-address statement refers to a symbol-table entry for the name. In Section 8.2, symbol-table entries were created as the declarations in a procedure were examined. The type in a declaration determines the width, i.e., the amount of storage, needed for the declared name. From the symbol-table information, a relative address can be determined for the name in a data area for the procedure. In Section 9.3, we outline implementations of static and stack allocation of data areas, and show how names in the intermediate representation can be converted into addresses in the target code.

If machine code is being generated, labels in three-address statements have to be converted to addresses of instructions. This process is analogous to the "backpatching" technique in Section 8.6. Suppose that labels refer to quadruple numbers in a quadruple array. As we scan each quadruple in turn we can deduce the location of the first machine instruction generated for that quadruple, simply by maintaining a count of the number of words used for the instructions generated so far. This count can be kept in the quadruple array (in an extra field), so if a reference such as $j$: `goto` $i$ is encountered, and $i$ is less than $j$, the current quadruple number, we may simply generate a jump instruction with the target address equal to the machine location of the first instruction in the code for quadruple $i$. If, however, the jump is forward, so $i$ exceeds $j$, we must store on a list for quadruple $i$ the location of the first machine instruction generated for quadruple $j$. Then, when we process quadruple $i$, we fill in the proper machine location for all instructions that are forward jumps to $i$.

### Instruction Selection

The nature of the instruction set of the target machine determines the difficulty of instruction selection. The uniformity and completeness of the instruction set are important factors. If the target machine does not support each data type in a uniform manner, then each exception to the general rule requires special handling.

Instruction speeds and machine idioms are other important factors. If we do not care about the efficiency of the target program, instruction selection is straightforward. For each type of three-address statement, we can design a code skeleton that outlines the target code to be generated for that construct. For example, every three-address statement of the form `x:=y+z`, where `x`, `y`, and `z` are statically allocated, can be translated into the code sequence

```
MOV y,R0   /* load y into register R0 */
ADD z,R0   /* add z to R0 */
MOV R0,x   /* store R0 into x */
```

Unfortunately, this kind of statement-by-statement code generation often produces poor code. For example, the sequence of statements

```
a := b + c
d := a + e
```

would be translated into

```
MOV b,R0
ADD c,R0
MOV R0,a
MOV a,R0
ADD e,R0
MOV R0,d
```

Here the fourth statement is redundant, and so is the third if `a` is not subsequently used.

The quality of the generated code is determined by its speed and size. A target machine with a rich instruction set may provide several ways of implementing a given operation. Since the cost differences between different implementations may be significant, a naive translation of the intermediate code may lead to correct, but unacceptably inefficient target code. For example, if the target machine has an "increment" instruction (`INC`), then the three-address statement `a := a+1` may be implemented more efficiently by the single instruction `INC a`, rather than by a more obvious sequence that loads a into a register, adds one to the register, and then stores the result back in a:

```
MOV   a, R0
ADD   #1, R0
MOV   R0, a
```

Instruction speeds are needed to design good code sequences but,

unfortunately, accurate timing information is often difficult to obtain. Deciding which machine-code sequence is best for a given three-address construct may also require knowledge about the context in which that construct appears. Tools for constructing instruction selectors are discussed in Section 9.12.

### Register Allocation

Instructions involving register operands are usually shorter and faster than those involving operands in memory. Therefore, efficient utilization of registers is particularly important in generating good code. The use of registers is often subdivided into two subproblems:

1. During *register allocation*, we select the set of variables that will reside in registers at a point in the program.
2. During a subsequent *register assignment* phase, we pick the specific register that a variable will reside in.

Finding an optimal assignment of registers to variables is difficult, even with single-register values. Mathematically, the problem is NP-complete. The problem is further complicated because the hardware and/or the operating system of the target machine may require that certain register-usage conventions be observed.

Certain machines require *register-pairs* (an even and next odd-numbered register) for some operands and results. For example, in the IBM System/370 machines integer multiplication and integer division involve register pairs. The multiplication instruction is of the form

```
M     x, y
```

where `x`, the multiplicand, is the even register of an even/odd register pair. The multiplicand value is taken from the odd register of the pair. The multiplier `y` is a single register. The product occupies the entire even/odd register pair.

The division instruction is of the form

```
D     x, y
```

where the 64-bit dividend occupies an even/odd register pair whose even register is `x`; `y` represents the divisor. After division, the even register holds the remainder and the odd register the quotient.

Now, consider the two three-address code sequences in Fig. 9.2(a) and (b), in which the only difference is the operator in the second statement. The shortest assembly-code sequences for (a) and (b) are given in Fig. 9.3.

`Ri` stands for register *i*. (`SRDA`[1] `R0,32` shifts the dividend into `R1` and clears `R0` so all bits equal its sign bit.) `L`, `ST`, and `A` stand for load, store and add, respectively. Note that the optimal choice for the register into which

[1] Shift Right Double Arithmetic.

```
t := a + b
t := t * c
t := t / d
```

(a)

```
t := a + b
t := t + c
t := t / d
```

(b)

**Fig. 9.2.** Two three-address code sequences.

```
L    R1, a
A    R1, b
M    R0, c
D    R0, d
ST   R1, t
```

(a)

```
L    R0, a
A    R0, b
A    R0, c
SRDA R0, 32
D    R0, d
ST   R1, t
```

(b)

**Fig. 9.3.** Optimal machine-code sequences.

a is to be loaded depends on what will ultimately happen to t. Strategies for register allocation are discussed in Section 9.7.

### Choice of Evaluation Order

The order in which computations are performed can affect the efficiency of the target code. Some computation orders require fewer registers to hold intermediate results than others, as we shall see. Picking a best order is another difficult, NP-complete problem. Initially, we shall avoid the problem by generating code for the three-address statements in the order in which they have been produced by the intermediate code generator.

### Approaches to Code Generation

Undoubtedly the most important criterion for a code generator is that it produce correct code. Correctness takes on special significance because of the number of special cases that a code generator might face. Given the premium on correctness, designing a code generator so it can be easily implemented, tested, and maintained is an important design goal.

Section 9.6 contains a straightforward code-generation algorithm that uses information about subsequent uses of an operand to generate code for a register machine. It considers each statement in turn, keeping operands in registers as long as possible. The output of such a code generator can be improved by peephole optimization techniques such as those discussed in Section 9.9.

Section 9.7 presents techniques to make better use of registers by considering the flow of control in the intermediate code. The emphasis is on

allocating registers for heavily used operands in inner loops.

Sections 9.10 and 9.11 present some tree-directed code-selection techniques that facilitate the construction of retargetable code generators. Versions of PCC, the portable C compiler, with such code generators have been moved to numerous machines. The availability of the UNIX operating system on a variety of machines owes much to the portability of PCC. Section 9.12 shows how code generation can be treated as a tree-rewriting process.

## 9.2 THE TARGET MACHINE

Familiarity with the target machine and its instruction set is a prerequisite for designing a good code generator. Unfortunately, in a general discussion of code generation it is not possible to describe the nuances of any target machine in sufficient detail to be able to generate good code for a complete language on that machine. In this chapter, we shall use as the target computer a register machine that is representative of several minicomputers. However, the code-generation techniques presented in this chapter have also been used on many other classes of machines.

Our target computer is a byte-addressable machine with four bytes to a word and $n$ general-purpose registers, `R0`, `R1`, . . . , `Rn-1`. It has two-address instructions of the form

*op source, destination*

in which *op* is an op-code, and *source* and *destination* are data fields. It has the following op-codes (among others):

| | |
|---|---|
| `MOV` | (move *source* to *destination*) |
| `ADD` | (add *source* to *destination*) |
| `SUB` | (subtract *source* from *destination*) |

Other instructions will be introduced as needed.

The source and destination fields are not long enough to hold memory addresses, so certain bit patterns in these fields specify that words following an instruction contain operands and/or addresses. The source and destination of an instruction are specified by combining registers and memory locations with address modes. In the following description, *contents*($a$) denotes the contents of the register or memory address represented by $a$.

The address modes together with their assembly-language forms and associated costs are as follows:

| MODE | FORM | ADDRESS | ADDED COST |
|---|---|---|---|
| *absolute* | `M` | `M` | 1 |
| *register* | `R` | `R` | 0 |
| *indexed* | `c(R)` | $c$ + *contents*(`R`) | 1 |
| *indirect register* | `*R` | *contents*(`R`) | 0 |
| *indirect indexed* | `*c(R)` | *contents*($c$ + *contents*(`R`)) | 1 |

A memory location M or a register R represents itself when used as a source or destination. For example, the instruction

```
MOV R0,M
```

stores the contents of register R0 into memory location M.

An address offset $c$ from the value in register R is written as $c$(R). Thus,

```
MOV 4(R0),M
```

stores the value

*contents*(4 + *contents*(R0))

into memory location M.

Indirect versions of the last two modes are indicated by prefix *. Thus,

```
MOV *4(R0),M
```

stores the value

*contents*(*contents*(4 + *contents*(R0)))

into memory location M.

A final address mode allows the source to be a constant:

| MODE | FORM | CONSTANT | ADDED COST |
|---|---|---|---|
| *literal* | #$c$ | $c$ | 1 |

Thus, the instruction

```
MOV #1,R0
```

loads the constant 1 into register R0.

### Instruction Costs

We take the cost of an instruction to be one plus the costs associated with the source and destination address modes (indicated as "added cost" in the table for address modes above). This cost corresponds to the length (in words) of the instruction. Address modes involving registers have cost zero, while those with a memory location or literal in them have cost one, because such operands have to be stored with the instruction.

If space is important, then we should clearly minimize instruction length. However, doing so has an important additional benefit. For most machines and for most instructions, the time taken to fetch an instruction from memory exceeds the time spent executing the instruction. Therefore, by minimizing the instruction length we also tend to minimize the time taken to perform the instruction as well.[2] Some examples follow.

[2] The cost criterion is meant to be instructive rather than realistic. Allowing a full word for an instruction simplifies the rule for determining the cost. A more accurate estimate of the time taken by an instruction would consider whether an instruction requires the value of an operand, as well

1. The instruction `MOV R0, R1` copies the contents of register `R0` into register `R1`. This instruction has cost one, since it occupies only one word of memory.

2. The (store) instruction `MOV R5, M` copies the contents of register `R5` into memory location `M`. This instruction has cost two, since the address of memory location `M` is in the word following the instruction.

3. The instruction `ADD #1, R3` adds the constant `1` to the contents of register 3, and has cost two, since the constant 1 must appear in the next word following the instruction.

4. The instruction `SUB 4(R0), *12(R1)` stores the value

   *contents*(*contents*(12 + *contents*(R1))) − *contents*(*contents*(4 + R0))

   into the destination `*12(R1)`. The cost of this instruction is three, since the constants 4 and 12 are stored in the next two words following the instruction.

Some of the difficulties in generating code for this machine can seen by considering what code to generate for a three-address statement of the form `a := b + c` where `b` and `c` are simple variables in distinct memory locations denoted by these names. This statement can be implemented by many different instruction sequences. Here are a few examples:

1.
```
MOV   b, R0
ADD   c, R0
MOV   R0, a
```
cost = 6

2.
```
MOV   b, a
ADD   c, a
```
cost = 6

Assuming `R0`, `R1`, and `R2` contain the addresses of `a`, `b`, and `c`, respectively, we can use:

3.
```
MOV   *R1, *R0
ADD   *R2, *R0
```
cost = 2

Assuming `R1` and `R2` contain the values of `b` and `c`, respectively, and that the value of `b` is not needed after the assignment, we can use:

4.
```
ADD   R2, R1
MOV   R1, a
```
cost = 3

We see that in order to generate good code for this machine, we must utilize its addressing capabilities efficiently. There is a premium on keeping the *l*- or *r*-value of a name in a register, if possible, if it is going to be used in the near future.

---

as its address (found with the instruction), to be fetched from memory.

## 9.3 RUN-TIME STORAGE MANAGEMENT

As we saw in Chapter 7, the semantics of procedures in a language determines how names are bound to storage during execution. Information needed during an execution of a procedure is kept in a block of storage called an activation record; storage for names local to the procedure also appears in the activation record.

In this section, we discuss what code to generate to manage activation records at run time. In Section 7.3, two standard storage-allocation strategies were presented, namely, static allocation and stack allocation. In static allocation, the position of an activation record in memory is fixed at compile time. In stack allocation, a new activation record is pushed onto the stack for each execution of a procedure. The record is popped when the activation ends. Later in this section, we consider how the target code of a procedure can refer to data objects in activation records.

As we saw in Section 7.2, an activation record for a procedure has fields to hold parameters, results, machine-status information, local data, temporaries and the like. In this section, we illustrate allocation strategies using the machine-status field to hold the return address and the field for local data. We assume the other fields are handled as discussed in Chapter 7.

Since run-time allocation and deallocation of activation records occurs as part of the procedure call and return sequences, we focus on the following three-address statements:

1. `call`,
2. `return`,
3. `halt`, and
4. `action`, a placeholder for other statements.

For example, the three-address code for procedures `c` and `p` in Fig. 9.4 contains just these kinds of statements. The size and layout of activation records are communicated to the code generator via the information about names that is in the symbol table. For clarity, we show the layout in Fig. 9.4, rather than the form of the symbol-table entries.

We assume that run-time memory is divided into areas for code, static data, and a stack, as in Section 7.2 (the additional area for a heap in that section is not used here).

### Static Allocation

Consider the code needed to implement static allocation. A `call` statement in the intermediate code is implemented by a sequence of two target-machine instructions. A `MOV` instruction saves the return address, and a `GOTO` transfers control to the target code for the called procedure:

```
MOV   #here + 20, callee.static_area
GOTO  callee.code_area
```

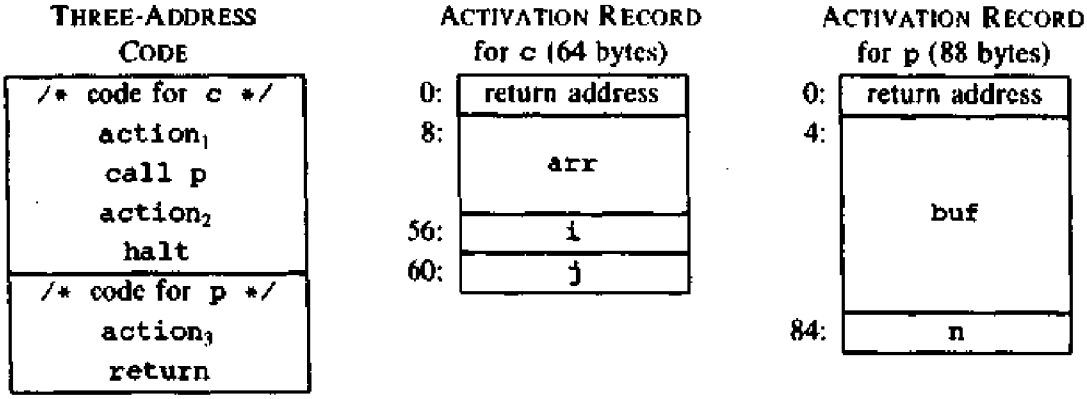


**Fig. 9.4.** Input to a code generator.

The attributes *callee.static_area* and *callee.code_area* are constants referring to the address of the activation record and the first instruction for the called procedure, respectively. The source *#here* + 20 in the MOV instruction is the literal return address; it is the address of the instruction following the GOTO instruction. (From the discussion in Section 9.2, the three constants plus the two instructions in the calling sequence cost 5 words or 20 bytes.)

The code for a procedure ends with a return to the calling procedure, except that the first procedure has no caller, so its final instruction is HALT, which presumably returns control to the operating system. A return from procedure *callee* is implemented by

```
GOTO *callee.static_area
```

which transfers control to the address saved at the beginning of the activation record.

**Example 9.1.** The code in Fig. 9.5 is constructed from the procedures c and p in Fig. 9.4. We use the pseudo-instruction ACTION to implement the statement action, which represents three-address code that is not relevant for this discussion. We arbitrarily start the code for these procedures at addresses 100 and 200, respectively, and assume that each ACTION instruction takes 20 bytes. The activation records for the procedures are statically allocated starting at location 300 and 364, respectively.

The instructions starting at address 100 implement the statements

```
action1; call p; action2; halt
```

of the first procedure c. Execution therefore starts with the instruction $\text{ACTION}_1$ at address 100. The MOV instruction at address 120 saves the return address 140 in the machine-status field, which is the first word in the activation record of p. The GOTO instruction at address 132 transfers control to the first instruction in the target code of the called procedure.

```
                        /* code for c */
100:  ACTION1
120:  MOV #140, 364     /* save return address 140 */
132:  GOTO 200          /* call p */
140:  ACTION2
160:  HALT
      ...

                        /* code for p */
200:  ACTION3
220:  GOTO *364         /* return to address saved in location 364 */
      ...

                        /* 300-363 hold activation record for c */
300:                    /* return address */
304:                    /* local data for c */
      ...

                        /* 364-451 hold activation record for p */
364:                    /* return address */
368:                    /* local data for p */
```

Fig. 9.5. Target code for the input in Fig. 9.4.

Since 140 was saved at address 364 by the call sequence above, *364 represents 140 when the GOTO statement at address 220 is executed. Control therefore returns to address 140 and execution of procedure c resumes. □

### Stack Allocation

Static allocation can become stack allocation by using relative addresses for storage in activation records. The position of the record for an activation of a procedure is not known until run time. In stack allocation, this position is usually stored in a register, so words in the activation record can be accessed as offsets from the value in this register. The indexed address mode of our target machine is convenient for this purpose.

Relative addresses in an activation record can be taken as offsets from any known position in the activation record, as we saw in Section 7.3. For convenience, we shall use positive offsets by maintaining in a register SP a pointer to the beginning of the activation record on top of the stack. When a procedure call occurs, the calling procedure increments SP and transfers control to the called procedure. After control returns to the caller, it decrements SP, thereby deallocating the activation record of the called procedure.[3]

[3] With negative offsets, we could have SP point to the end of the stack and have the called procedure increment SP.

The code for the first procedure initializes the stack by setting SP to the start of the stack area in memory:

```
MOV #stackstart, SP          /* initialize the stack */
code for the first procedure
HALT                         /* terminate execution */
```

A procedure call sequence increments SP, saves the return address, and transfers control to the called procedure:

```
ADD #caller.recordsize, SP
MOV #here + 16, *SP          /* save return address */
GOTO callee.code_area
```

The attribute *caller.recordsize* represents the size of an activation record, so the ADD instruction leaves SP pointing to the beginning of the next activation record. The source *#here* + 16 in the MOV instruction is the address of the instruction following the GOTO; it is saved in the address pointed to by SP.

The return sequence consists of two parts. The called procedure transfers control to the return address using

```
GOTO *0(SP)    /* return to caller */
```

The reason for using *0(SP) in the GOTO instruction is that we need two levels of indirection: 0(SP) is the address of the first word in the activation record, and *0(SP) is the return address saved there.

The second part of the return sequence is in the caller, which decrements SP, thereby restoring SP to its previous value. That is, after the subtraction SP points to the beginning of the activation record of the caller:

```
SUB #caller.recordsize, SP
```

A broader discussion of calling sequences and the tradeoffs in the division of labor between the calling and called procedures appears in Section 7.3.

**Example 9.2.** The program in Fig. 9.6 is a condensation of the three-address code for the Pascal program discussed in Section 7.1. Procedure q is recursive, so more than one activation of q can be alive at the same time.

Suppose that the sizes of the activation records for procedures s, p, and q have been determined at compile time to be *ssize*, *psize* and *qsize*, respectively. The first word in each activation record will hold a return address. We arbitrarily assume that the code for these procedures starts at addresses 100, 200, and 300, respectively, and that the stack starts at 600. The target code for the program in Fig. 9.6 is as follows:

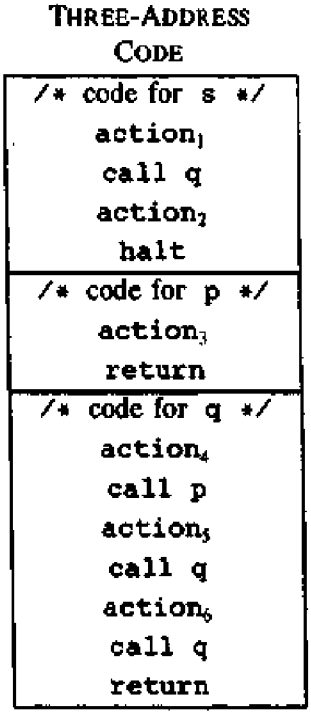


**Fig. 9.6.** Three-address code to illustrate stack allocation

```
                        /* code for s */
100:  MOV #600, SP      /* initialize the stack */
108:  ACTION1
128:  ADD #ssize, SP    /* call sequence begins */
136:  MOV #152, *SP     /* push return address */
144:  GOTO 300          /* call q */
152:  SUB #ssize, SP    /* restore SP */
160:  ACTION2
180:  HALT
      . . .
                        /* code for p */
200:  ACTION3
220:  GOTO *0(SP)       /* return */
      . . .
                        /* code for q */
300:  ACTION4           /* conditional jump to 456 */
320:  ADD #qsize, SP
328:  MOV #344, *SP     /* push return address */
336:  GOTO 200          /* call p */
344:  SUB #qsize, SP
352:  ACTION5
372:  ADD #qsize, SP
```

```
380:  MOV #396, *SP    /* push return address */
388:  GOTO 300         /* call q */
396:  SUB #qsize, SP
404:  ACTION6
424:  ADD #qsize, SP
432:  MOV #448, *SP    /* push return address */
440:  GOTO 300         /* call q */
448:  SUB #qsize, SP
456:  GOTO *0(SP)      /* return */
      . . .

600:                   /* stack starts here */
```

We assume that $\texttt{ACTION}_4$ contains a conditional jump to the address 456 of the return sequence from q; otherwise, the recursive procedure q is condemned to call itself forever. In an example below, we consider an execution of the program in which the first call of q does not return immediately, but all subsequent calls do.

If *ssize*, *psize* and *qsize* are 20, 40, and 60, respectively, then SP is initialized to 600, the starting address of the stack, by the first instruction at address 100. SP holds 620 just before control transfers from s to q, because *ssize* is 20. Subsequently, when q calls p, the instruction at address 320 increments SP to 680, where the activation record for p begins; SP reverts to 620 after control returns to q. If the next two recursive calls of q return immediately, the maximum value of SP during this execution is 680. Note, however, that the last stack location used is 739, since the activation record for q starting at location 680 extends for 60 bytes. □

### Run-Time Addresses for Names

The storage-allocation strategy and the layout of local data in an activation record for a procedure determine how the storage for names is accessed. In Chapter 8, we assumed that a name in a three-address statement is really a pointer to a symbol-table entry for the name. This approach has a significant advantage; it makes the compiler more portable, since the front end need not be changed even if the compiler is moved to a different machine where a different run-time organization is needed (e.g., the display can be kept in registers instead of memory). On the other hand, generating the specific sequence of access steps while generating intermediate code can be of significant advantage in an optimizing compiler, since it lets the optimizer take advantage of details it would not even see in the simple three-address statement.

In either case, names must eventually be replaced by code to access storage locations. We thus consider some elaborations of the simple three-address copy statement x := 0. After the declarations in a procedure are processed, suppose the symbol-table entry for x contains a relative address 12 for x. First consider the case in which x is in a statically allocated area beginning at address *static*. Then the actual run-time address of x is *static* + 12. Although

the compiler can eventually determine the value of *static* + 12 at compile time, the position of the static area may not be known when intermediate code to access the name is generated. In that case, it makes sense to generate three-address code to "compute" *static* + 12, with the understanding that this computation will be carried out during the code-generation phase, or possibly by the loader, before the program runs. The assignment `x := 0` then translates into

```
static[12] := 0
```

If the static area starts at address 100, the target code for this statement is

```
MOV #0, 112
```

On the other hand, suppose our language is one like Pascal and that a display is used to access nonlocal names, as discussed in Section 7.4. Suppose also that the display is kept in registers, and that `x` is local to an active procedure whose display pointer is in register `R3`. Then we may translate the copy `x := 0` into the three-address statements

```
t1 := 12 + R3
*t1 := 0
```

in which $t_1$ contains the address of `x`. This sequence can be implemented by the single machine instruction

```
MOV #0, 12(R3)
```

Notice that the value in register `R3` cannot be determined at compile time.

## 9.4 BASIC BLOCKS AND FLOW GRAPHS

A graph representation of three-address statements, called a flow graph, is useful for understanding code-generation algorithms, even if the graph is not explicitly constructed by a code-generation algorithm. Nodes in the flow graph represent computations, and the edges represent the flow of control. In Chapter 10, we use the flow graph of a program extensively as a vehicle to collect information about the intermediate program. Some register-assignment algorithms use flow graphs to find the inner loops where a program is expected to spend most of its time.

### Basic Blocks

A *basic block* is a sequence of consecutive statements in which flow of control enters at the beginning and leaves at the end without halt or possibility of branching except at the end. The following sequence of three-address statements forms a basic block:

```
t1 := a * a
t2 := a * b
```

```
t3 := 2 * t2
t4 := t1 + t3
t5 := b * b
t6 := t4 + t5
```
(9.1)

A three-address statement `x := y+z` is said to *define* `x` and to *use* (or *reference*) `y` and `z`. A name in a basic block is said to be *live* at a given point if its value is used after that point in the program, perhaps in another basic block.

The following algorithm can be used to partition a sequence of three-address statements into basic blocks.

**Algorithm 9.1.** Partition into basic blocks.

*Input.* A sequence of three-address statements.

*Output.* A list of basic blocks with each three-address statement in exactly one block.

*Method.*

1. We first determine the set of *leaders*, the first statements of basic blocks. The rules we use are the following.
   i) The first statement is a leader.
   ii) Any statement that is the target of a conditional or unconditional goto is a leader.
   iii) Any statement that immediately follows a goto or conditional goto statement is a leader.
2. For each leader, its basic block consists of the leader and all statements up to but not including the next leader or the end of the program. □

**Example 9.3.** Consider the fragment of source code shown in Fig. 9.7; it computes the dot product of two vectors `a` and `b` of length 20. A list of three-address statements performing this computation on our target machine is shown in Fig. 9.8.

```
begin
    prod := 0;
    i := 1;
    do begin
        prod := prod + a[i] * b[i];
        i := i + 1
    end
    while i <= 20
end
```

**Fig. 9.7.** Program to compute dot product.

Let us apply Algorithm 9.1 to the three-address code in Fig. 9.8 to determine its basic blocks. Statement (1) is a leader by rule (i) and statement (3) is a leader by rule (ii), since the last statement can jump to it. By rule (iii) the statement following (12) (recall that Fig. 9.13 is just a fragment of a program) is a leader. Therefore, statements (1) and (2) form a basic block. The remainder of the program beginning with statement (3) forms a second basic block. □

```
(1)   prod := 0
(2)   i := 1
(3)   t1 := 4 * i
(4)   t2 := a [ t1 ]          /* compute a[i] */
(5)   t3 := 4 * i
(6)   t4 := b [ t3 ]          /* compute b[i] */
(7)   t5 := t2 * t4
(8)   t6 := prod + t5
(9)   prod := t6
(10)  t7 := i + 1
(11)  i := t7
(12)  if i <= 20 goto (3)
```

**Fig. 9.8.** Three-address code computing dot product.

### Transformations on Basic Blocks

A basic block computes a set of expressions. These expressions are the values of the names live on exit from the block. Two basic blocks are said to be *equivalent* if they compute the same set of expressions.

A number of transformations can be applied to a basic block without changing the set of expressions computed by the block. Many of these transformations are useful for improving the quality of code that will be ultimately generated from a basic block. In the next chapter, we show how a global code "optimizer" tries to use such transformations to rearrange the computations in a program in an effort to reduce the overall running time or space requirement of the final target program. There are two important classes of local transformations that can be applied to basic blocks; these are the structure-preserving transformations and the algebraic transformations.

### Structure-Preserving Transformations

The primary structure-preserving transformations on basic blocks are:

1. common subexpression elimination
2. dead-code elimination
3. renaming of temporary variables
4. interchange of two independent adjacent statements

Let us now examine these transformations in a little more detail. For the time being, we assume basic blocks have no arrays, pointers, or procedure calls.

1. *Common subexpression elimination.* Consider the basic block

```
a := b + c
b := a - d
c := b + c                    (9.2)
d := a - d
```

The second and fourth statements compute the same expression, namely, b+c−d, and hence this basic block may be transformed into the equivalent block

```
a := b + c
b := a - d
c := b + c                    (9.3)
d := b
```

Note that although the first and third statements in (9.2) and (9.3) appear to have the same expression on the right, the second statement redefines b. Therefore, the value of b in the third statement is different from the value of b in the first, and the first and third statements do not compute the same expression.

2. *Dead-code elimination.* Suppose x is dead, that is, never subsequently used, at the point where the statement x := y+z appears in a basic block. Then this statement may be safely removed without changing the value of the basic block.

3. *Renaming temporary variables.* Suppose we have a statement t := b+c, where t is a temporary. If we change this statement to u := b+c, where u is a new temporary variable, and change all uses of this instance of t to u, then the value of the basic block is not changed. In fact, we can always transform a basic block into an equivalent block in which each statement that defines a temporary defines a new temporary. We call such a basic block a *normal-form* block.

4. *Interchange of statements.* Suppose we have a block with the two adjacent statements

```
t1 := b + c
t2 := x + y
```

Then we can interchange the two statements without affecting the value of the block if and only if neither x nor y is $t_1$ and neither b nor c is $t_2$. Notice that a normal-form basic block permits all statement interchanges that are possible.

### Algebraic Transformations

Countless algebraic transformations can be used to change the set of expressions computed by a basic block into an algebraically equivalent set. The useful ones are those that simplify expressions or replace expensive operations by cheaper ones. For example, statements such as

```
x := x + 0
```

or

```
x := x * 1
```

can be eliminated from a basic block without changing the set of expressions it computes. The exponentiation operator in the statement

```
x := y ** 2
```

usually requires a function call to implement. Using an algebraic transformation, this statement can be replaced by the cheaper, but equivalent statement

```
x := y * y
```

Algebraic transformations are discussed in more detail in Section 9.9 on peephole optimization and in Section 10.3 on optimization of basic blocks.

### Flow Graphs

We can add the flow-of-control information to the set of basic blocks making up a program by constructing a directed graph called a *flow graph*. The nodes of the flow graph are the basic blocks. One node is distinguished as *initial*; it is the block whose leader is the first statement. There is a directed edge from block $B_1$ to block $B_2$ if $B_2$ can immediately follow $B_1$ in some execution sequence; that is, if

1. there is a conditional or unconditional jump from the last statement of $B_1$ to the first statement of $B_2$, or
2. $B_2$ immediately follows $B_1$ in the order of the program, and $B_1$ does not end in an unconditional jump.

We say that $B_1$ is a *predecessor* of $B_2$, and $B_2$ is a *successor* of $B_1$.

**Example 9.4.** The flow graph of the program of Fig. 9.7 is shown in Fig. 9.9. $B_1$ is the initial node. Note that in the last statement, the jump to statement (3) has been replaced by an equivalent jump to the beginning of block $B_2$. □

### Representation of Basic Blocks

Basic blocks can be represented by a variety of data structures. For example, after partitioning the three-address statements by Algorithm 9.1, each basic block can be represented by a record consisting of a count of the number of quadruples in the block, followed by a pointer to the leader (first quadruple)

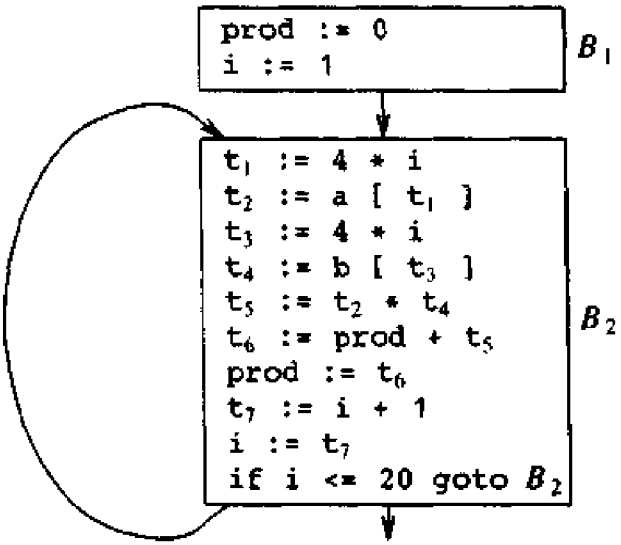


**Fig. 9.9.** Flow graph for program.

of the block, and by the lists of predecessors and successors of the block. An alternative is to make a linked list of the quadruples in each block. Explicit references to quadruple numbers in jump statements at the end of basic blocks can cause problems if quadruples are moved during code optimization. For example, if the block $B_2$ running from statements (3) through (12) in the intermediate code of Fig. 9.9 were moved elsewhere in the quadruple array or were shrunk, the (3) in `if i <= 20 goto (3)` would have to be changed. Thus, we prefer to make jumps point to blocks rather than quadruples, as we have done in Fig. 9.9.

It is important to note that an edge of the flow graph from block $B$ to block $B'$ does not specify the conditions under which control flows from $B$ to $B'$. That is, the edge does not tell whether the conditional jump at the end of $B$ (if there is a conditional jump there) goes to the leader of $B'$ when the condition is satisfied or when the condition is not satisfied. That information can be recovered when needed from the jump statement in $B$.

### Loops

In a flow graph, what is a loop, and how does one find all loops? Most of the time, it is easy to answer these questions. For example, in Fig. 9.9 there is one loop, consisting of block $B_2$. The general answers to these questions, however, are a bit subtle, and we shall examine them in detail in the next chapter. For the present, it is sufficient to say that a loop is a collection of nodes in a flow graph such that

1. All nodes in the collection are *strongly connected*; that is, from any node in the loop to any other, there is a path of length one or more, wholly within the loop, and

2. The collection of nodes has a unique *entry*, that is, a node in the loop such that the only way to reach a node of the loop from a node outside the loop is to first go through the entry.

A loop that contains no other loops is called an *inner* loop.

## 9.5 NEXT-USE INFORMATION

In this section, we collect next-use information about names in basic blocks. If the name in a register is no longer needed, then the register can be assigned to some other name. This idea of keeping a name in storage only if it will be used subsequently can be applied in a number of contexts. We used it in Section 5.8 to assign space for attribute values. The simple code generator in the next section applies it to register assignment. As a final application, we consider the assignment of storage for temporary names.

### Computing Next Uses

The *use* of a name in a three-address statement is defined as follows. Suppose three-address statement $i$ assigns a value to $x$. If statement $j$ has $x$ as an operand, and control can flow from statement $i$ to $j$ along a path that has no intervening assignments to $x$, then we say statement $j$ *uses* the value of $x$ computed at $i$.

We wish to determine for each three-address statement $x := y\ op\ z$ what the next uses of $x$, $y$, and $z$ are. For the present, we do not concern ourselves with uses outside the basic block containing this three-address statement but we may, if we wish, attempt to determine whether or not there is such a use by the live-variable analysis technique of Chapter 10.

Our algorithm to determine next uses makes a backward pass over each basic block. We can easily scan a stream of three-address statements to find the ends of basic blocks as in Algorithm 9.1. Since procedures can have arbitrary side effects, we assume for convenience that each procedure call starts a new basic block.

Having found the end of a basic block, we scan backwards to the beginning, recording (in the symbol table) for each name $x$ whether $x$ has a next use in the block and if not, whether it is live on exit from that block. If the data-flow analysis discussed in Chapter 10 has been done, we know which names are live on exit from each block. If no live-variable analysis has been done, we can assume all nontemporary variables are live on exit, to be conservative. If the algorithms generating intermediate code or optimizing the code permit certain temporaries to be used across blocks, these too must be considered live. It would be a good idea to mark any such temporaries, so we do not have to consider all temporaries live.

Suppose we reach three-address statement $i$: $x := y\ op\ z$ in our backward scan. We then do the following.

1. Attach to statement $i$ the information currently found in the symbol table

regarding the next use and liveness of x, y, and z.[4]

2. In the symbol table, set x to "not live" and "no next use."

3. In the symbol table, set y and z to "live" and the next uses of y and z to *i*. Note that the order of steps (2) and (3) may not be interchanged because x may be y or z.

If three-address statement *i* is of the form x := y or x := *op* y, the steps are the same as above, ignoring z.

### Storage for Temporary Names

Although it may be useful in an optimizing compiler to create a distinct name each time a temporary is needed (see Chapter 10 for justification), space has to be allocated to hold the values of these temporaries. The size of the field for temporaries in the general activation record of Section 7.2 grows with the number of temporaries.

We can, in general, pack two temporaries into the same location if they are not live simultaneously. Since almost all temporaries are defined and used within basic blocks, next-use information can be applied to pack temporaries. For temporaries that are used across blocks, Chapter 10 discusses the data-flow analysis needed to compute liveness.

We can allocate storage locations for temporaries by examining each in turn and assigning a temporary to the first location in the field for temporaries that does not contain a live temporary. If a temporary cannot be assigned to any previously created location, add a new location to the data area for the current procedure. In many cases, temporaries can be packed into registers rather than memory locations, as in the next section.

For example, the six temporaries in the basic block (9.1) can be packed into two locations. These locations correspond to $t_1$ and $t_2$ in:

```
t1 := a * a
t2 := a * b
t2 := 2 * t2
t1 := t1 + t2
t2 := b * b
t1 := t1 + t2
```

## 9.6 A SIMPLE CODE GENERATOR

The code-generation strategy in this section generates target code for a sequence of three-address statement. It considers each statement in turn, remembering if any of the operands of the statement are currently in registers, and taking advantage of that fact if possible. For simplicity, we assume that

[4] If x is not live, then this statement can be deleted; such transformations are considered in Section 9.8.

for each operator in a statement there is a corresponding target-language operator. We also assume that computed results can be left in registers as long as possible, storing them only (a) if their register is needed for another computation or (b) just before a procedure call, jump, or labeled statement.[5]

Condition (b) implies that everything must be stored just before the end of a basic block.[6] The reason we must do so is that, after leaving a basic block, we may be able to go to several different blocks, or we may go to one particular block that can be reached from several others. In either case, we cannot, without extra effort, assume that a datum used by a block appears in the same register no matter how control reached that block. Thus, to avoid a possible error, our simple code-generation algorithm stores everything when moving across basic-block boundaries as well as when procedure calls are made. Later we consider ways to hold some data in registers across block boundaries.

We can produce reasonable code for a three-address statement `a := b + c` if we generate the single instruction `ADD Rj, Ri` with cost one, leaving the result `a` in register `Ri`. This sequence is possible only if register `Ri` contains `b`, `Rj` contains `c`, and `b` is not live after the statement; that is, `b` is not used after the statement.

If `Ri` contains `b` but `c` is in a memory location (called `c` for convenience), we can generate the sequence

```
ADD    c, Ri          cost = 2
```

or

```
MOV    c, Rj
ADD    Rj, Ri         cost = 3
```

provided `b` is not subsequently live. The second sequence becomes attractive if this value of `c` is subsequently used, as we can then take its value from register `Rj`. There are many more cases to consider, depending on where `b` and `c` are currently located and depending on whether the current value of `b` is subsequently used. We must also consider the cases where one or both of `b` and `c` is a constant. The number of cases that need to be considered further increases if we assume that the operator `+` is commutative. Thus, we see that code generation involves examining a large number of cases, and which case should prevail depends on the context in which a three-address statement is seen.

---

[5] However, to produce a *symbolic dump*, which makes available the values of memory locations and registers in terms of the source program's names for these values, it may be more convenient to have programmer-defined variables (but not necessarily compiler-generated temporaries) stored immediately upon calculation, should a program error suddenly cause a precipitous interrupt and exit.

[6] Note we are not assuming that the quadruples were actually partitioned into basic blocks by the compiler; the notion of a basic block is useful conceptually in any event.

### Register and Address Descriptors

The code-generation algorithm uses descriptors to keep track of register contents and addresses for names.

1. A register descriptor keeps track of what is currently in each register. It is consulted whenever a new register is needed. We assume that initially the register descriptor shows that all registers are empty. (If registers are assigned across blocks, this would not be the case.) As the code generation for the block progresses, each register will hold the value of zero or more names at any given time.

2. An address descriptor keeps track of the location (or locations) where the current value of the name can be found at run time. The location might be a register, a stack location, a memory address, or some set of these, since when copied, a value also stays where it was. This information can be stored in the symbol table and is used to determine the accessing method for a name.

### A Code-Generation Algorithm

The code-generation algorithm takes as input a sequence of three-address statements constituting a basic block. For each three-address statement of the form `x := y` *op* `z` we perform the following actions:

1. Invoke a function *getreg* to determine the location `L` where the result of the computation `y` *op* `z` should be stored. `L` will usually be a register, but it could also be a memory location. We shall describe *getreg* shortly.

2. Consult the address descriptor for `y` to determine `y'`, (one of) the current location(s) of `y`. Prefer the register for `y'` if the value of `y` is currently both in memory and a register. If the value of `y` is not already in `L`, generate the instruction `MOV y',L` to place a copy of `y` in `L`.

3. Generate the instruction `OP z',L` where `z'` is a current location of `z`. Again, prefer a register to a memory location if `z` is in both. Update the address descriptor of `x` to indicate that `x` is in location `L`. If `L` is a register, update its descriptor to indicate that it contains the value of `x`, and remove `x` from all other register descriptors.

4. If the current values of `y` and/or `z` have no next uses, are not live on exit from the block, and are in registers, alter the register descriptor to indicate that, after execution of `x := y` *op* `z`, those registers no longer will contain `y` and/or `z`, respectively.

If the current three-address statement has a unary operator, the steps are analogous to those above, and we omit the details. An important special case is a three-address statement `x := y`. If `y` is in a register, simply change the register and address descriptors to record that the value of `x` is now found only in the register holding the value of `y`. If `y` has no next use and is not

live on exit from the block, the register no longer holds the value of y.

If y is only in memory, we could in principle record that the value of x is in the location of y, but this option would complicate our algorithm, since we could not then change the value of y without preserving the value of x. Thus, if y is in memory we use *getreg* to find a register in which to load y and make that register the location of x.

Alternatively, we can generate a MOV y, x instruction, which would be preferable if the value of x has no next use in the block. It is worth noting that most, if not all, copy instructions will be eliminated if we use the block-improving and copy-propagation algorithm of Chapter 10.

Once we have processed all three-address statements in the basic block, we store, by MOV instructions, those names that are live on exit and not in their memory locations. To do this we use the register descriptor to determine what names are left in registers, the address descriptor to determine that the same name is not already in its memory location, and the live variable information to determine whether the name is to be stored. If no live-variable information has been computed by data-flow analysis among blocks, we must assume all user-defined names are live at the end of the block.

### The Function *getreg*

The function *getreg* returns the location L to hold the value of x for the assignment x := y *op* z. A great deal of effort can be expended in implementing this function to produce a perspicacious choice for L. In this section, we discuss a simple, easy-to-implement scheme based on the next-use information collected in the last section.

1. If the name y is in a register that holds the value of no other names (recall that copy instructions such as x := y could cause a register to hold the value of two or more variables simultaneously), and y is not live and has no next use after execution of x := y *op* z, then return the register of y for L. Update the address descriptor of y to indicate that y is no longer in L.

2. Failing (1), return an empty register for L if there is one.

3. Failing (2), if x has a next use in the block, or *op* is an operator, such as indexing, that requires a register, find an occupied register R. Store the value of R into a memory location (by MOV R, M) if it is not already in the proper memory location M, update the address descriptor for M, and return R. If R holds the value of several variables, a MOV instruction must be generated for each variable that needs to be stored. A suitable occupied register might be one whose datum is referenced furthest in the future, or one whose value is also in memory. We leave the exact choice unspecified, since there is no one proven best way to make the selection.

4. If x is not used in the block, or no suitable occupied register can be found, select the memory location of x as L.

A more sophisticated *getreg* function would also consider the subsequent uses of x and the commutativity of the operator *op* in determining the register to hold the value of x. We leave such extensions of *getreg* as exercises.

**Example 9.5.** The assignment `d := (a-b) + (a-c) + (a-c)` might be translated into the following three-address code sequence

```
t := a - b
u := a - c
v := t + u
d := v + u
```

with d live at the end. The code-generation algorithm given above would produce the code sequence shown in Fig. 9.10 for this three-address statement sequence. Shown alongside are the values of the register and address descriptors as code generation progresses. Not shown in the address descriptor is the fact that a, b, and c are always in memory. We also assume that t, u and v, being temporaries, are not in memory unless we explicitly store their values with a MOV instruction.

| STATEMENTS | CODE GENERATED | REGISTER DESCRIPTOR | ADDRESS DESCRIPTOR |
|---|---|---|---|
| | | registers empty | |
| t := a - b | MOV a, R0 | R0 contains t | t in R0 |
| | SUB b, R0 | | |
| u := a - c | MOV a, R1 | R0 contains t | t in R0 |
| | SUB c, R1 | R1 contains u | u in R1 |
| v := t + u | ADD R1, R0 | R0 contains v | u in R1 |
| | | R1 contains u | v in R0 |
| d := v + u | ADD R1, R0 | R0 contains d | d in R0 |
| | MOV R0, d | | d in R0 and memory |

**Fig. 9.10.** Code sequence.

The first call of *getreg* returns R0 as the location in which to compute t. Since a is not in R0, we generate instructions `MOV a,R0` and `SUB b,R0`. We now update the register descriptor to indicate that R0 contains t.

Code generation proceeds in this manner until the last three-address statement `d := v+u` has been processed. Note that R1 becomes empty because u has no next use. We then generate `MOV R0,d` to store the live variable d at the end of the block.

The cost of the code generated in Fig. 9.10 is 12. We could reduce this to

11 by generating `MOV R0,R1` immediately after the first instruction and removing the instruction `MOV a,R1`, but to do so requires a more sophisticated code-generation algorithm. The reason for the savings is that it is cheaper to load `R1` from `R0` than from memory. □

### Generating Code for Other Types of Statements

Indexing and pointer operations in three-address statements are handled in the same manner as binary operations. The table in Fig. 9.11 shows the code sequences generated for the indexed assignment statements `a := b[i]` and `a[i] := b`, assuming `b` is statically allocated.

| STATEMENT | i IN REGISTER Ri | | i IN MEMORY Mi | | i IN STACK | |
|---|---|---|---|---|---|---|
| | CODE | COST | CODE | COST | CODE | COST |
| a:=b[i] | MOV b(Ri),R | 2 | MOV Mi,R<br>MOV b(R),R | 4 | MOV Si(A),R<br>MOV b(R),R | 4 |
| a[i]:=b | MOV b,a(Ri) | 3 | MOV Mi,R<br>MOV b,a(R) | 5 | MOV Si(A),R<br>MOV b,a(R) | 5 |

**Fig. 9.11.** Code sequences for indexed assignments.

The current location of `i` determines the code sequence. Three cases are covered depending on whether `i` is in register `Ri`, whether `i` is in memory location `Mi`, or whether `i` is on the stack at offset `Si` and the pointer to the activation record for `i` is in register `A`. The register `R` is the register returned when the function *getreg* is called. For the first assignment, we would prefer to leave `a` in register `R` if `a` has a next use in the block and register `R` is available. In the second assignment, we assume `a` is statically allocated.

The table in Fig. 9.12 shows the code sequences generated for the pointer assignments `a := *p` and `*p := a`. Here, the current location of `p` determines the code sequence.

| STATEMENT | p IN REGISTER Rp | | p IN MEMORY Mp | | p IN STACK | |
|---|---|---|---|---|---|---|
| | CODE | COST | CODE | COST | CODE | COST |
| a:=*p | MOV *Rp,a | 2 | MOV Mp,R<br>MOV *R,R | 3 | MOV Sp(A),R<br>MOV *R,R | 3 |
| *p:=a | MOV a,*Rp | 2 | MOV Mp,R<br>MOV a,*R | 4 | MOV a,R<br>MOV R,*Sp(A) | 4 |

**Fig. 9.12.** Code sequences for pointer assignments.

Three cases are covered depending on whether `p` is initially in register `Rp`, whether `p` is in memory location `Mp`, or whether `p` is on the stack at offset `Sp`

and the pointer to the activation record for p is in register A. The register R is the register returned when the function *getreg* is called. In the second assignment, we assume a is statically allocated.

### Conditional Statements

Machines implement conditional jumps in one of two ways. One way is to branch if the value of a designated register meets one of the six conditions: negative, zero, positive, nonnegative, nonzero, and nonpositive. On such a machine a three-address statement such as `if x<y goto z` can be implemented by subtracting y from x in register R, and then jumping to z if the value in register R is negative.

A second approach, common to many machines, uses a set of *condition codes* to indicate whether the last quantity computed or loaded into a register is negative, zero, or positive. Often a compare instruction (CMP in our machine) has the desirable property that it sets the condition code without actually computing a value. That is, `CMP x, y` sets the condition code to positive if x > y, and so on. A conditional-jump machine instruction makes the jump if a designated condition $<$, $=$, $>$, $\leq$, $\neq$, or $\geq$ is met. We use the instruction `CJ<= z` to mean "jump to z if the condition code is negative or zero." For example, `if x<y goto z` could be implemented by

```
CMP   x, y
CJ<   z
```

If we are generating code for a machine with condition codes it is useful to maintain a condition-code descriptor as we generate code. This descriptor tells the name that last set the condition code, or the pair of names compared, if the condition code was last set that way. Thus we could implement

```
x := y + z
if x < 0 goto z
```

by

```
MOV   y, R0
ADD   z, R0
MOV   R0, x
CJ<   z
```

if we were aware that the condition code was determined by x after `ADD z, R0`.

## 9.7 REGISTER ALLOCATION AND ASSIGNMENT

Instructions involving only register operands are shorter and faster than those involving memory operands. Therefore, efficient utilization of registers is important in generating good code. This section presents various strategies for deciding what values in a program should reside in registers (register

allocation) and in which register each value should reside (register assignment).

One approach to register allocation and assignment is to assign specific values in an object program to certain registers. For example, a decision can be made to assign base addresses to one group of registers, arithmetic computation to another, the top of the run-time stack to a fixed register, and so on.

This approach has the advantage that it simplifies the design of a compiler. Its disadvantage is that, applied too strictly, it uses registers inefficiently; certain registers may go unused over substantial portions of code, while unnecessary loads and stores are generated. Nevertheless, it is reasonable in most computing environments to reserve a few registers for base registers, stack pointers and the like, and to allow the remaining registers to be used by the compiler as it sees fit.

### Global Register Allocation

The code-generation algorithm in Section 9.6 used registers to hold values for the duration of a single basic block. However, all live variables were stored at the end of each block. To save some of these stores and corresponding loads, we might arrange to assign registers to frequently used variables and keep these registers consistent across block boundaries (*globally*). Since programs spend most of their time in inner loops, a natural approach to global register assignment is to try to keep a frequently used value in a fixed register throughout a loop. For the time being, assume that we know the loop structure of a flow graph, and that we know what values computed in a basic block are used outside that block. The next chapter covers techniques for computing this information.

One strategy for global register allocation is to assign some fixed number of registers to hold the most active values in each inner loop. The selected values may be different in different loops. Registers not already allocated may be used to hold values local to one block as in Section 9.6. This approach has the drawback that the fixed number of registers is not always the right number to make available for global register allocation. Yet the method is simple to implement and it has been used in Fortran H, the optimizing Fortran compiler for the IBM-360 series machines (Lowry and Medlock [1969]).

In languages like C and Bliss, a programmer can do some register allocation directly by using register declarations to keep certain values in registers for the duration of a procedure. Judicious use of register declarations can speed up many programs, but a programmer should not do register allocation without first profiling the program.

### Usage Counts

A simple method for determining the savings to be realized by keeping variable $x$ in a register for the duration of loop $L$ is to recognize that in our machine model we save one unit of cost for each reference to $x$ if $x$ is in a